Setup

Install it on the GPU server.

For: Engineers

One chapter of the handbook. The full handbook, with every chapter, is handbook.pdf in the same folder.

This guide installs secretary-keeper on one GPU server with Docker Compose. Follow the steps in order. Each step ends with a check; don't move on until it passes.

1. What you need

Item	Minimum	Notes
GPUs	1 × 80 GB (H100/H200) for `judge`, 1 smaller GPU for `embed` + `rerank`	Add 1 for `extract`, and 4 Blackwell or 8 Hopper for `escalate` (optional)
NVIDIA driver + Container Toolkit	Recent enough for your vLLM image	`docker run --rm --gpus all nvidia/cuda:12.4.1-base-ubuntu22.04 nvidia-smi` must list your GPUs
Docker Engine + Compose v2	Compose 2.20 or later	`docker compose version`
Disk	About 400 GB	Mostly model weights
A machine with internet access	Temporary	To download images and weights before moving them in

2. Bring images and weights into the environment

On a machine **with** internet access:

```
# Images: pin the exact tags you will write into .env
docker pull vllm/vllm-openai:<VLLM_TAG>
docker pull ghcr.io/huggingface/text-embeddings-inference:<TEI_TAG>
docker pull ghcr.io/open-webui/open-webui:<OPENWEBUI_TAG>
docker pull pgvector/pgvector:pg17
docker pull nginx:1.27-alpine
# Tag must equal APP_VERSION in .env, so Compose uses this image instead of trying to build one.
docker build -t secretary-keeper/policy-api:0.1.0 policy_api    # needs PyPI or your internal
mirror
docker save -o secretary-keeper-images.tar vllm/vllm-openai:<VLLM_TAG> ... secretary-
keeper/policy-api:0.1.0

# Weights (see models/README.md for the list)
hf download openai/gpt-oss-120b --local-dir models/gpt-oss-120b
# ...embedding and reranker...
scripts/verify_models.sh record
```

Move the tar file, the repo and the `models/` folder into the environment. Then:

```
docker load -i secretary-keeper-images.tar
scripts/verify_models.sh check
```

Check: `verify_models.sh check` prints "All model files match their recorded hashes."

3. Configure

```
cp .env.example .env
for k in POSTGRES_PASSWORD POLICY_API_KEY LINK_SIGNING_KEY LLM_API_KEY OPENWEBUI_SECRET_KEY; do
  sed -i "s/^$k=.*/$k=$(openssl rand -hex 32)/" .env
done
chmod 600 .env
```

Then edit `.env`:

- `VLLM_TAG`, `TEI_TAG`, `OPENWEBUI_TAG`: the exact tags you loaded.
- `PUBLIC_BASE_URL`: the address people's browsers use for downloads, e.g. `https://policy.plant.local:8100`.
- `POLICY_ID_PATTERN`: if your IDs don't look like `ABC-123`.
- `POLICY_SOURCE_DIR`: the folder holding your policy files.

Assign GPUs. `compose.yaml` names none, so you choose them for this server:

```
python3 scripts/setup_gpus.py
```

It lists your GPUs, asks which ones each model uses, checks the plan (for example that `judge` has about 80 GB and no two big models share a GPU), writes `compose.gpu.yaml` and sets `COMPOSE_FILE`, `JUDGE_TP` and `ESCALATE_TP` in `.env`. Run it again whenever the hardware changes. To do it by hand instead, follow the comments in `compose.gpu.example.yaml`, which also has sizing notes per model.

If your embedding model's output size is not 768, change `vector(768)` in `db/init.sql` now, before the database is created.

Check: `docker compose config --quiet` prints nothing.

4. Start the stack

```
docker compose up -d db embed rerank judge
docker compose logs -f judge          # wait for "Application startup complete" (can take 10–20
minutes)
docker compose up -d policy-api open-webui proxy
```

Optional models:

```
docker compose --profile escalate up -d escalate   # then set
ESCALATE_URL=http://escalate:8000/v1 in .env
docker compose --profile extract up -d extract     # then set EXTRACT_URL=http://extract:8000/v1
in .env
docker compose up -d policy-api                     # restart to pick up .env changes
```

Check: `docker compose ps` shows every service as running or healthy.

5. Prove the environment is contained

```
scripts/check_containment.sh
```

Check: it ends with `PASS contained`. Each service it checks must first reach an internal service (proving the probe works) and then fail to reach the internet. If it says FAIL, stop and fix it before loading real policies. See security.md.

6. Load policies

Policy files can be `.md`, `.txt`, `.docx` or `.pdf`. A short header block at the top of each file gives the details the system can't guess:

```
---
id: SAF-112
title: Maintenance Access
owner: Maintenance Manager
parent: POL-007
subsets: [Safety]
version: 1
---
```

Without a header, the ID is taken from the file name or the first lines, and the version counts up. Groups (`subsets`) must be in the header.

```
docker compose run --rm policy-api python -m app.ingest /policies
# Also use the extract model to find references the regex misses (slower):
docker compose run --rm policy-api python -m app.ingest /policies --llm-refs
```

Check: it prints `ingested <ID> v<N>` for each file and lists any skipped files with the reason.

7. Set up Open WebUI

Open `http://<server>:3000`. The first account you create becomes the admin. Then:

1. **Turn off sign-up.** Set `OPENWEBUI_ENABLE_SIGNUP=false` in `.env` and run `docker compose up -d open-webui`. Add users from the Admin Panel.
2. **Add the tool.** Workspace → Tools → + → paste all of `openwebui/policy_impact_tool.py` → Save.
3. **Set the tool's settings (Valves).** Open the tool's gear icon. Set `API_KEY` to the `POLICY_API_KEY` from `.env`. Leave `API_URL` as `http://policy-api:8000`.

4. **Create the assistant model.** Workspace → Models → +: - Name: `Policy Impact Assistant`; base model: `judge` - System prompt: paste `openwebui/system_prompt.md` - Tools: tick **Policy Impact Checker** - Advanced Params → **Function Calling: Native** - Save, and make it the default model for users (Admin Panel → Settings → Models).
5. **Hide the bare `judge` model** from regular users, so everyone uses the assistant.

Menu names move between Open WebUI releases. If a step doesn't match, look for the same setting nearby. The Open WebUI tool reads attached files from `__files__`; if uploads stop working after an Open WebUI upgrade, check that first (operations.md).

Check: in a new chat, ask *"Which Safety policies are affected by the change to POL-007?"* and attach the new version. You get an answer with a download link, and the link opens an Excel file.

8. Before real use

- [ ] `scripts/check_containment.sh` passes
- [ ] Evaluation on at least 10 past changes meets your recall target (evaluation.md)
- [ ] Security checklist done (security.md)
- [ ] Two or three staff have tried it using only how-to.md, without help